Interrupts:

- Interrupt is the method of creating ~~halt~~ a temporary halt during program execution & allows peripheral devices to access the microprocessor.

or:

The meaning of 'interrupt' is to breaks the sequence of operation, while the CPU is executing a program on 'interrupt' breaks the normal sequence of execution of instruction, diverts its execution to some other program called ~~interrup~~ interrupt service routine. (ISR), After executing the ISR, the control is transferred back again to the main program.

Interrupts of 8086:

ISR. It is a special program to instruct the microprocessor on how to handle the interrupts.

* Types of Interrupts:

interrupts of 8086

- Hardware interrupts
  - mastable interrupts
    - eg.) INTR
  - Non-mastable interrupts (NMI)
- software interrupts
  - 256 Types of S/W interrupts.
    - Ex: INT type.

① Hardware Interrupts:

- Hardware interrupt is caused by any peripheral device by sending a signal through a specified pin to the microprocessor.
- The 8086 has two hardware interrupts pins. i.e NMI and INTR.

- Non-maskable Interrupts:-

- It is non-maskable interrupts.
- It is a single non-maskable interrupts pin (NMI) having higher priority than the maskable interrupts request pin (INTR) - interrupt request
- Non-maskable interrupt can not be disabled or we can't ignore the non-maskable interrupts
- i.e It is the highest priority interrupt

- For this non-maskable interrupt
std location
- IP (Instruction pointer) is loaded from 0008H
- code segment (CS) is loaded from 0000AH
- It is a type-2 interrupt

ii) INTR:

INTR (interrupt request) provides a signal request & is activated by I/O port
- This can be masked or delayed maskable interrupt
- It is a level triggered interrupt & can receive any interrupt type.

- The INTR interrupt is activated by I/O port. If the interrupt is enabled & TISNI is disabled then the µP first complete the current execution & sends '0' (zero) on INTA pin twice.
- The first '0' means INTA informs the external device to get ready & during second '0' the µP receives the 8 bit, say X, from the programmable interrupt controller

Software Interrupts.

- This software interrupts can be generated by inserting the instruction "INT" within the program.
- Their are 256 software interrupts available in 8086 µP

Format:

INT type;

type ranges from 00 to FF H (H - Hexadecimal value)

- starting address range from 00000H to 003FFH

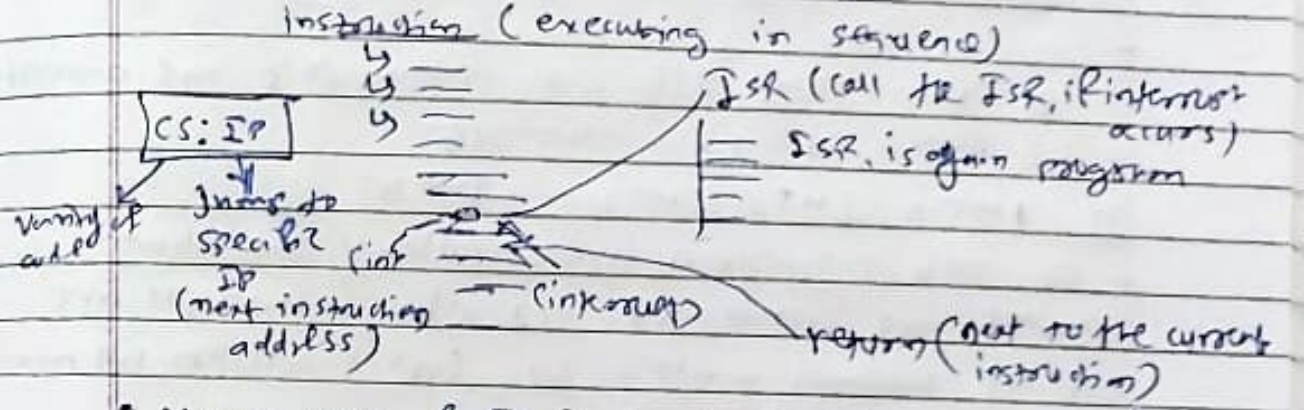
- These are 2 byte instructions

INT type number

opcode immediate value

ISR - interrupt service routine - program

- variety of interrupt → how to handle that program is nothing but ISR



- Always value of IP is incremented every time, it will execute the next instruction
- Always Interrupt is executed after current instruction execution
- Interrupt :- Distribution

- Hardware interrupt :- and software interrupt
- software interrupts
- Hardware interrupt :-
- HW interrupts are not return. Pin's are allotted for HW interrupt,
- 8086 - two HW interrupts

a) NMI :

- Non maskable, we can't ignore such interrupts microprocessor need to implement this on priority basis
- vectored interrupt : means address already known to this interrupt

b) INTR :

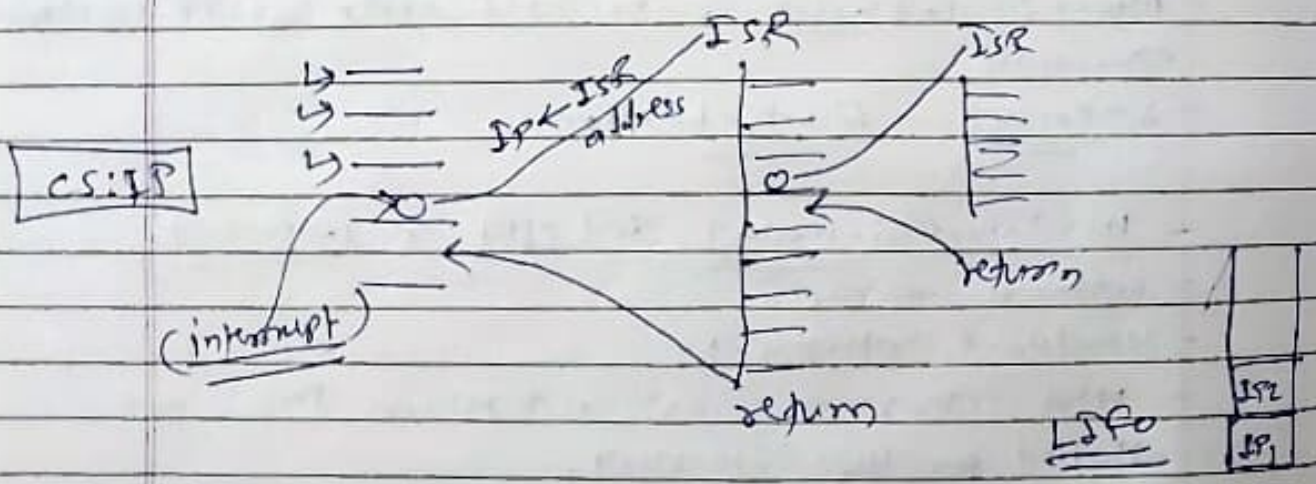
- maskable interrupt we can ignore such interrupt need to implement but on low priority.

- Non-vectored: such type of interrupt doesnot know the address, so microprocessor will ask the address with the help of INTA (interrupt acknowledgement)

Software interrupt

-

- 0-255 - Total 256 s/w interrupt's are available
- ie Type 0, Type 1 --- Type 255
or INT 0, INT 1, INT 2 --- INT 255
- So, 256 programs are available for above stated. 256 interrupts. coz s/w interrupts are already ~~return~~ written by sys. (written by means ISR is written in memory)
- Now, How to execute these interrupts



- If you want return to next instruction then I have to store address of next instruction into stack.

- (256 ISR): value is stored in IVT
so IVT will store address of ISR
- By default table address is 00000

- To store single address of 1 ISR

16 bit	16 bit
CS	IP

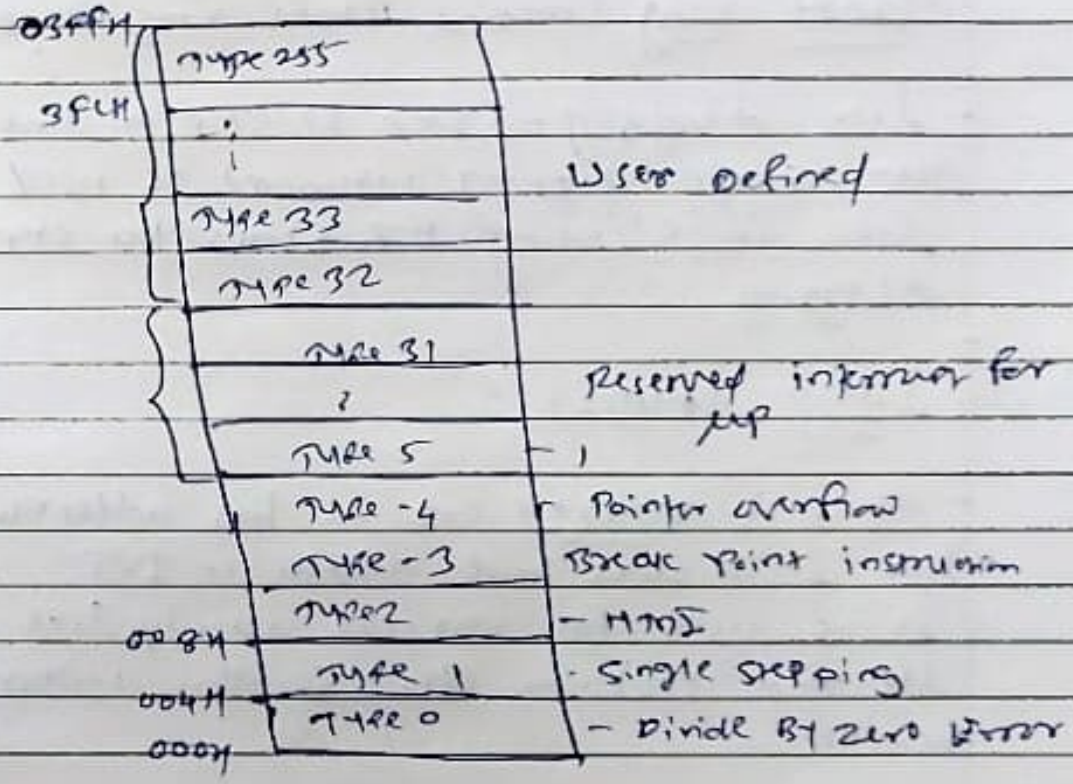
4 Byte

So, To store 1 ISR address 4-byte (2 for CS & 2 for IP)

- Total, $256 \times 4 = 1024$ byte

IVT = 1KB

IVT Table:



Type - 0 - 4 bytes | CS : IP | - 2 byte for each

↳ 00000H to 00003H

Type 255 ↳ 00007FH to 003FFH

i) INT0: Divide Error
- Is used for Divide error
i.e a/b ⇒ if b=0
a/0 ⇒

So in order to find out such type of error µP uses the INT0 interrupt like in java we are using try & catch block

ii) INT1: Single stepping - with the help of Traff Flag (means Traff flag is used)

- like debugging ⇒ step by step or line by line then single stepping interrupt is used like in 'C' using F7 shows you step by step debegging

iii) INT2: (NMI)

- as it is vectored coz it has address in IVT but INTA does have address in IVT
- It is used for various activity like µP overheat µP fan problem etc. so this interrupt will handle.

iv) INT3: (Break point)

-

iv) INT4: (interrupt on overflow)
- used when you are using small register & you are trying to do multiplication operation, & that result will not stored in that register, then in that case, your µP will jump in to INTA interrupt

- Then this interrupt will handle how to handle such operation.

- INT-5 to INT 31: Reserved for μP

- INT-32 to INT 255: user defined means you can create your own interrupt & store in INT.

hf